

Kubernetes Cluster Networking

Since you're learning **KCNA as a beginner**, let's understand **Cluster Networking** in a very simple way.

1. What is Cluster Networking?

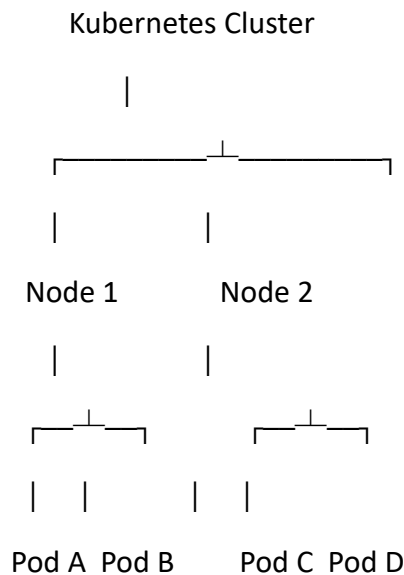
Cluster Networking is the system that allows **Pods, Nodes, Services, and other Kubernetes components to communicate with each other.**

In simple words:

 **Cluster Networking = How everything inside a Kubernetes cluster talks to each other.**

2. Imagine a Kubernetes Cluster

Suppose we have:



These Pods need to communicate:

Pod A → Pod B

Pod A → Pod C

Pod C → Pod D

Cluster networking makes this communication possible.

3. Main Networking Requirements ★

For KCNA, remember these important Kubernetes networking concepts.

1 Pod-to-Pod communication

Pods should be able to communicate with other Pods.

Pod A —————→ Pod B

Even if they are on different Nodes:

Node 1 Node 2

Pod A —————→ Pod B

2 Node-to-Pod communication

A Node should be able to communicate with Pods.

Node

|



Pod

3 Pod-to-Service communication

Pods often communicate with applications through a **Service**.

Pod A

|



Service

|



Pod B

The Service provides a stable way to reach a group of Pods.

4. Every Pod Gets an IP Address ★

One important Kubernetes networking concept:

Each Pod gets its own IP address.

Example:

Frontend Pod

IP: 10.1.0.5

Backend Pod

IP: 10.1.0.6

They can communicate:

10.1.0.5 —————→ 10.1.0.6

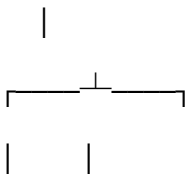
5. Containers Inside the Same Pod

Containers inside the **same Pod share the Pod's network namespace and IP address.**

Example:

Pod

IP: 10.1.0.5



Container A Container B

Both containers can communicate with each other using:

localhost

🧠 **Remember:**

Containers in the same Pod share the Pod's network.

6. Pod-to-Pod Networking

Suppose:

Pod A

10.1.0.5

needs to communicate with:

Pod B

10.1.0.6

The Kubernetes networking system allows:

Pod A

10.1.0.5

|

| Network



Pod B

10.1.0.6

The goal is for Pods to communicate without requiring special NAT tricks for normal Pod-to-Pod traffic.

7. Pods on Different Nodes ★

This is very important.

Imagine:

Node 1

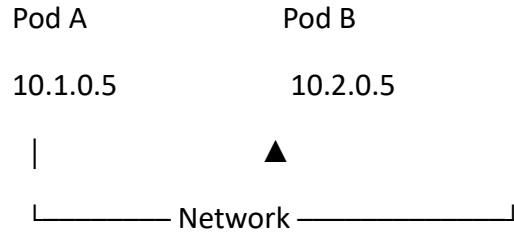
Node 2

|

|

|

|



Pod A should be able to communicate with Pod B even though they are on different Nodes.

The **cluster network** provides the connectivity needed between them.

8. What Provides Kubernetes Networking?

Kubernetes relies on a **network plugin** to implement Pod networking.

This is often called a:

CNI (Container Network Interface) plugin

Examples include:

- **Cilium**
- **Calico**
- **Flannel**

The CNI/plugin is responsible for implementing networking functionality for Pods.

Easy memory:

 **CNI = Helps provide container/Pod networking**

9. What is CNI?

CNI stands for:

Container Network Interface

It is a standard for configuring network interfaces for containers.

Very simply:

Pod created

|



CNI / Network Plugin



└─ Provides networking

└─ Configures network interface

10. Service Networking

Pods can be temporary.

For example:

Pod A

10.1.0.5

might be deleted and replaced by:

Pod B

10.1.0.10

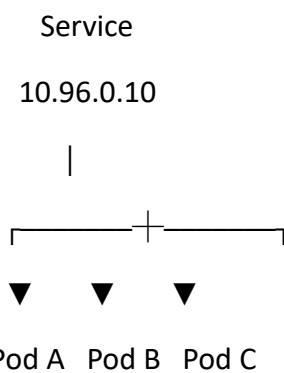
The IP changed.

So applications shouldn't normally depend directly on individual Pod IPs.

Instead, Kubernetes provides a:

Service

Example:



The Service provides a stable way to access the backend Pods.

11. ClusterIP

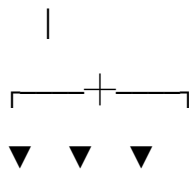
The most common Service type for internal cluster communication is:

ClusterIP

Example:

Backend Service

ClusterIP: 10.96.0.20



Pod A Pod B Pod C

Other Pods can communicate with the Service instead of worrying about individual Pod IPs.

12. kube-proxy

You learned about **kube-proxy** when studying Kubernetes components.

kube-proxy is involved in implementing Service networking on Nodes.

Simple picture:

Pod



Service



kube-proxy / networking rules



Backend Pod

🧠 **Memory:**

🚦 **kube-proxy** = Helps direct Service traffic to the appropriate Pods

13. DNS in Kubernetes ★

Kubernetes also provides **DNS-based service discovery**.

Instead of remembering:

10.96.0.20

an application can generally use a Service name.

For example:

backend-service

So:

Frontend Pod

|



backend-service

|



Backend Pods

This makes communication easier.

14. NetworkPolicy

This connects directly to the topic you just learned.

Cluster networking provides connectivity.

NetworkPolicy controls/restricts that connectivity.

Think:

Cluster Networking

|



"Can Pods communicate?"

|



NetworkPolicy

|



"Which traffic is allowed?"

Easy difference:

 **Networking = Communication**

 **NetworkPolicy = Control of communication**

15. Real-Life Example

Imagine a hospital.

Reception

|



Doctor's Room

|



Laboratory

Networking provides the roads:

Reception ———→ Doctor ———→ Lab

NetworkPolicy acts like security gates:

Reception → Doctor 

Reception → Lab 

Doctor → Lab 

So:

Networking creates communication paths.

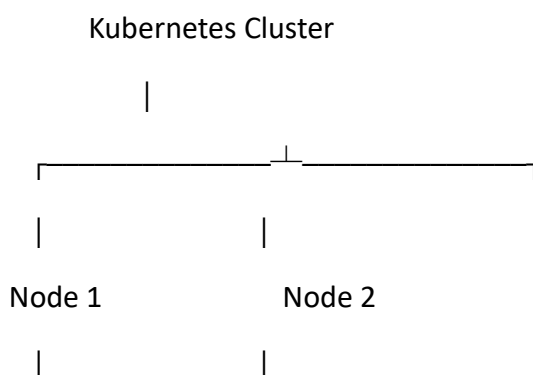
NetworkPolicy controls which paths are allowed.

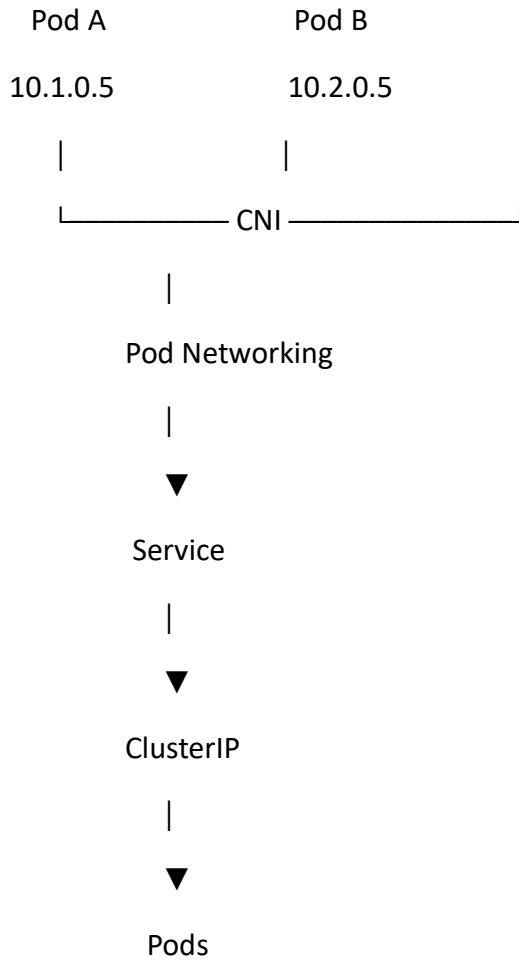
16. Important Kubernetes Networking Components

For your KCNA revision:

Component	Easy Meaning
Pod IP	IP address assigned to a Pod
CNI	Provides/configures Pod networking
Service	Stable way to access Pods
ClusterIP	Internal Service IP
kube-proxy	Helps implement Service traffic routing
DNS	Allows service discovery by name
NetworkPolicy	Controls allowed network traffic

Complete Picture





And:

NetworkPolicy



Controls the traffic

★ KCNA Revision Notes

Cluster Networking

- **Cluster Networking = Communication between Kubernetes components and workloads.**
- Pods receive **IP addresses**.
- Pods can communicate with other Pods.

- Pods can communicate across different Nodes.
- Containers in the same Pod share the Pod's network namespace/IP.
- **CNI/network plugins** provide Pod networking.
- **Services** provide a stable way to access Pods.
- **ClusterIP** provides an internal Service IP.
- **kube-proxy** helps implement Service traffic routing.
- **Kubernetes DNS** provides service discovery.
- **NetworkPolicy** controls allowed network traffic.

Super Easy Memory

CNI → Pod Networking 

Pod IP → Pod Address 

Service → Stable Access 

ClusterIP → Internal Service IP

DNS → Find Services by Name 

kube-proxy → Service Traffic 

NetworkPolicy → Control Traffic 

One-line definition:

Kubernetes Cluster Networking enables Pods, Nodes, Services, and other workloads to communicate within and, where configured, outside the cluster.